

CMSE 801 - Day 19



Exploring the properties of an agent-based model
for modeling forest fires

November 6, 2024

General Course Announcements

- Homework 4 is available and due **November 15 by 11:59pm**.
- I reviewed all of the project proposals, if I had any concerns, you should have received an email from me.
- Make sure you're working on your project!
 - You'll be presenting project drafts during the Day 23 class period (**November 20**).

Questions/comments from Day 19 PCA

- Some folks ran into issues getting the plurality voting functions coded up
 - This requires some careful array indexing and thoughtful conditional statements.
- Other folks were able to get the plurality stuff working, but ran into issues with calculating the fraction of last place votes.

Let's generate our ballots!

```
In [1]: # Generating the data table
import numpy as np
import random
ballot_list=[]
for i in range(29):
    ballot_list.append(['a','b','c'])
for i in range(1):
    ballot_list.append(['a','c','b'])
for i in range(23):
    ballot_list.append(['b','a','c'])
for i in range(3):
    ballot_list.append(['b','c','a'])
for i in range(26):
    ballot_list.append(['c','b','a'])
for i in range(18):
    ballot_list.append(['c','a','b'])
# make the data into a 2D numpy array
ballot_list=np.array(ballot_list)
# use np.random.shuffle to mix up the order just for fun and authenticity
np.random.shuffle(ballot_list)
print(ballot_list[:5])
```

```
[['c' 'a' 'b']  
 ['a' 'b' 'c']  
 ['a' 'b' 'c']  
 ['b' 'a' 'c']  
 ['a' 'b' 'c']]
```

Let's take a look at couple
possible solutions for the
plurality voting

```
In [2]: ## Option 1, go through every ballot with a loop and see who was selected  
def plurality_1(bl):  
    candidate_count= np.zeros(3)  
    for i in range(len(bl)):  
        if bl[i,0] == 'a':  
            candidate_count[0]+=1  
        elif bl[i,0]=='b':  
            candidate_count[1]+=1  
        else:  
            candidate_count[2]+=1  
    print("First choice votes for each candidate:\n",candidate_count)  
    return np.argmax(candidate_count)
```

```
In [3]: print("Which candidate won?",plurality_1(ballot_list))
```

First choice votes for each candidate:

[30. 26. 44.]

Which candidate won? 2

```
In [4]: ## Option 2: create an array of all the first choice votes, then make a
def plurality_2(bl):
    first_choice_votes=np.array(bl[:,0])
    candidate_count= np.zeros(3)
    candidate_count[0]=len(first_choice_votes[first_choice_votes == 'a'])
    candidate_count[1]=len(first_choice_votes[first_choice_votes == 'b'])
    candidate_count[2]=len(first_choice_votes[first_choice_votes == 'c'])
    print("First choice votes for each candidate:\n",candidate_count)
    return np.argmax(candidate_count)
```

```
In [5]: print("Which candidate won?",plurality_2(ballot_list))
```

```
First choice votes for each candidate:
[30. 26. 44.]
Which candidate won? 2
```

What if we want to know the fraction of people that chose a candidate *last*?

```
In [6]: def lastplace(bl, c):  
        # We use array indexing to get the last place votes  
        # Then we create mask for those values that match the candidate  
        # We sum that mask because "True" is equal to "1"  
        # and then divide the result by the total number of ballots  
        frac_last = (bl[:, -1] == c).sum() / len(ballot_list)  
        return frac_last  
  
print(lastplace(ballot_list, 'a'))  
print(lastplace(ballot_list, 'b'))  
print(lastplace(ballot_list, 'c'))
```

0.29

0.19

0.52

But we could have also used our previous function!

```
In [7]: def plurality_last_fraction(bl):  
        first_choice_votes=np.array(bl[:,-1])  
        candidate_count= np.zeros(3)  
        candidate_count[0]=len(first_choice_votes[first_choice_votes == 'a'])  
        candidate_count[1]=len(first_choice_votes[first_choice_votes == 'b'])  
        candidate_count[2]=len(first_choice_votes[first_choice_votes == 'c'])  
        print("Last choice votes for each candidate:\n",candidate_count)  
        return candidate_count[0]/len(bl), candidate_count[1]/len(bl), cand:
```

```
In [8]: print("Last choice fractions?",plurality_last_fraction(ballot_list))
```

Last choice votes for each candidate:

[29. 19. 52.]

Last choice fractions? (0.29, 0.19, 0.52)

Using `np.where` to find values locations and to mask and filter data

```
In [13]: my_array=np.array([1,2,3,5,1])  
print("Result of where:",np.where(my_array==2))  
print("Results of using where to mask data:",my_array[np.where(my_array==2)])
```

```
Result of where: (array([1]),)
```

```
Results of using where to mask data: [2]
```

Let's explore this with the ballot list, which is 2D

```
In [14]: candidate_c = np.where(ballot_list == 'c')
print("Result of where:\n", candidate_c)
```

Result of where:

```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13,
        14, 15, 16,
        17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 3
        1, 32, 33,
        34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 4
        8, 49, 50,
        51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 6
        5, 66, 67,
        68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 8
        2, 83, 84,
        85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 9
        9]), array([0, 2, 2, 2, 2, 2, 0, 2, 2, 1, 2, 2, 0, 2, 0, 2, 2,
        2, 0, 2, 2, 0,
        0, 2, 0, 0, 2, 2, 0, 2, 0, 2, 2, 0, 2, 2, 2, 2, 0, 2, 0,
        0, 0, 0,
        0, 2, 0, 0, 2, 0, 2, 2, 2, 0, 0, 0, 0, 2, 0, 2, 0, 0, 0,
        0, 2, 2,
        2, 2, 0, 0, 2, 2, 2, 2, 0, 2, 0, 0, 2, 0, 1, 2, 2, 0, 2,
        2, 1, 0,
        0, 0, 2, 0, 0, 0, 2, 2, 2, 1, 2, 0]))
```

```
In [15]: ballot_list[candidate_c]
```

Out[15]:

```
array(['c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c',  
      'c', 'c',  
        'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c',  
      'c', 'c',  
        'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c',  
      'c', 'c',  
        'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c',  
      'c', 'c',  
        'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c',  
      'c', 'c',  
        'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c',  
      'c', 'c',  
        'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c',  
      'c', 'c',  
        'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c',  
      'c', 'c',  
        'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c'], dtype='<U  
1')
```

Goals for today

- Continue manipulating 2D NumPy arrays to store data as well as to build and run models
- Finish off the forest fire model.
 - Functions from last time are provided
 - You need to build a function to analyze the state of the board
 - Read, comment, and understand some code that runs one simulation
 - You need to complete a some that runs many simulations for a series of starting tree fractions
- The work you do today will allow you to explore the properties of the model you built
- You'll also be able to explore if changing the initial state of the board has a significant impact on the results

Looking forward - Day 20 PCA: An introduction to "Object-oriented programming" (OOP) in Python

- You'll explore the concept of "object-oriented programming"
- You'll think about how you've already leveraged this programming paradigm in how you've been using Python
- You'll be introduced to Python "classes" which is what Python calls "objects"

